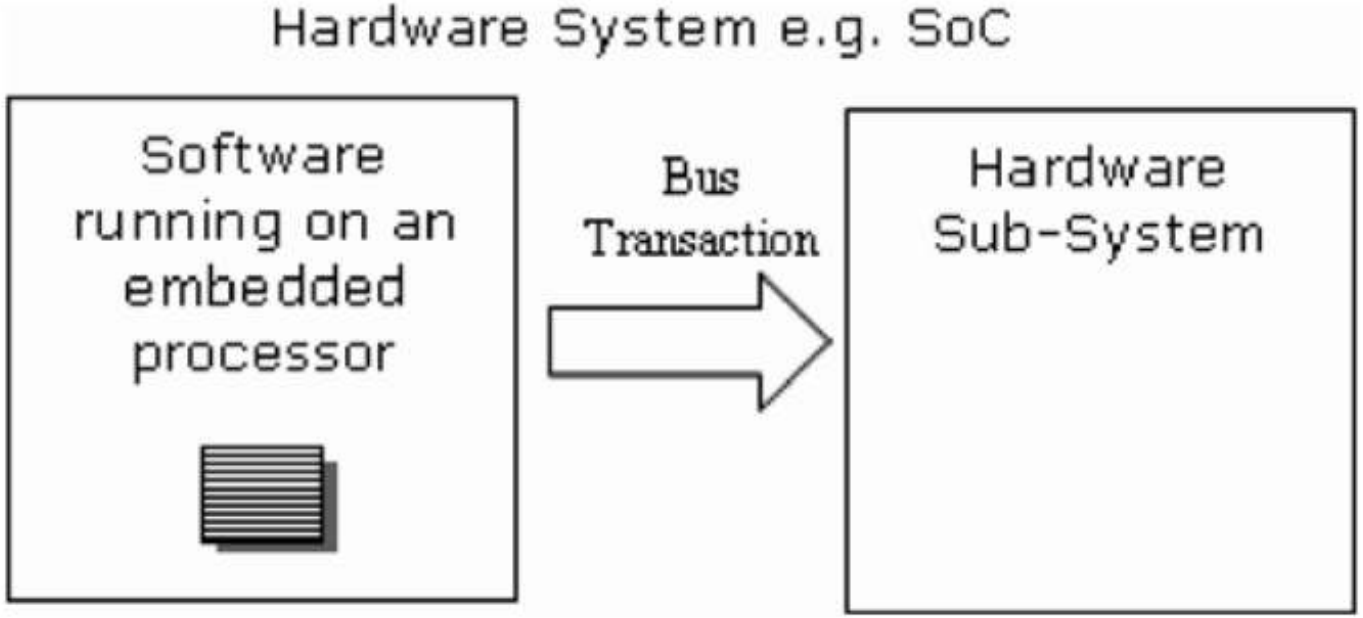
Overview

Overwiev

HW/SF Interface

- Software accesses hardware by converting software **instructions** into **hardware transactions**

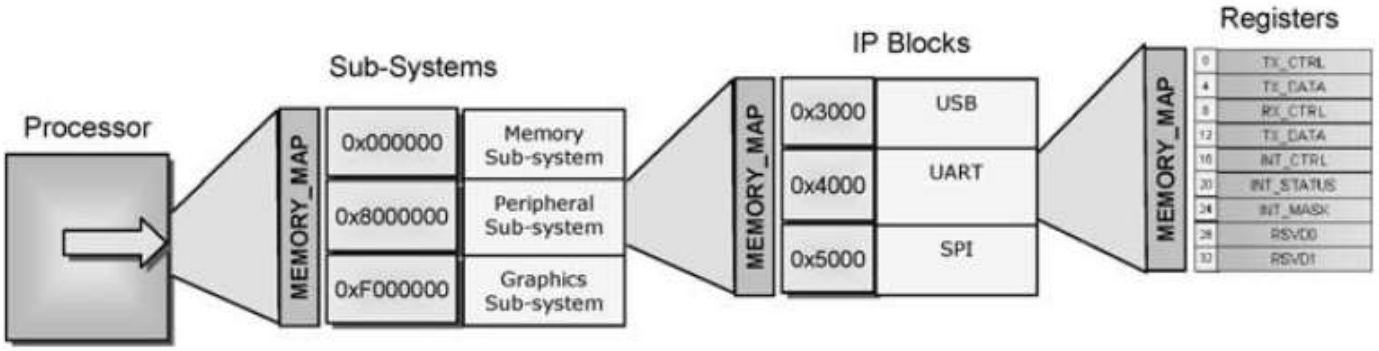


Hardware Configuration and Control Using Software

- Software can take following actions:
- **Resetting** the hardware to a known state
- **Configuration** of the hardware to perform a specific function
- Getting **status** of the hardware
- Reading/writing **data** to/from the hardware.
- Sample power control hardware element:
  - 0x0: Turn Power on
  - 0x1: Turn Power off
  - 0x2: Set Power to standby
  - 0x3: Set Power to idle
- Those generic storage mechanisms that can be accessed by the processor, via a memory-mapped transaction, can be generically described as **Software Accessible Hardware Elements (SAHEs)**.
- This may be implemented in hardware with constructs such as memories, registers, or bitfields.

Software Perspective I

- From the processor point of view the **system** can be viewed as an **accessible address space**.
- This address space is generally an **ordered layout** or map of the hardware.



- Usually the best practice in embedded software design is to ensure optimal processor performance by **reducing** the number of **transactions** between hardware and software.
- The processor typically works with a certain **data bus size**, - 32 bit/64 bit.

Software Perspective II

- The least addressable unit (**LAU**) - the minimum data access. The LAU imposes a constraint on the HW/SW interface.
- How do we write 2 bits on 8-bit LAU?
- The first option is to assign this SAHE a dedicated address space, meaning that the other 30 bits will **not be used**, or probably made "reserved."
- Pro: **ease** of access
- Con: An increase in the amount of address **decoding** is required.
- Con: Increase in the number of **transactions**
- The second option is to **"pack"** as many SAHEs as possible into a shared address.
- What happens when just one of the SAHEs needs to be written?
- 1. Either the software retains a **copy** of the hardware SAHEs

Software Perspective III

- 2. The software must perform a **read–modify–write** type of transaction.
- This type of transaction requires some latency between the initial read and the final updating write; this introduces an opportunity for **non-deterministic errors.**
- In the case where the SAHE is **bigger** than the maximum addressable unit, then accessing this HW element may involve **several accesses** to write or read the data.

Interrupts

- Another important part of the HW/SW interface is the ability of the hardware to **notify** the software when something has occurred. This is typically done using a **hardware interrupt**.
- From the software perspective, interrupts are **asynchronous breaks** in program flow that occur as a result of events **outside** the running program
- Software typically handles interrupts through an **interrupt service routine (ISR)** which can become extremely **cumbersome** for a large system.
- The ISR has the possibility of **disabling** interrupts in the hardware at many different levels to help manage **priorities**.
- **Interrupt latency** - the response time between an interrupt being asserted in the hardware and the corresponding ISR being executed.

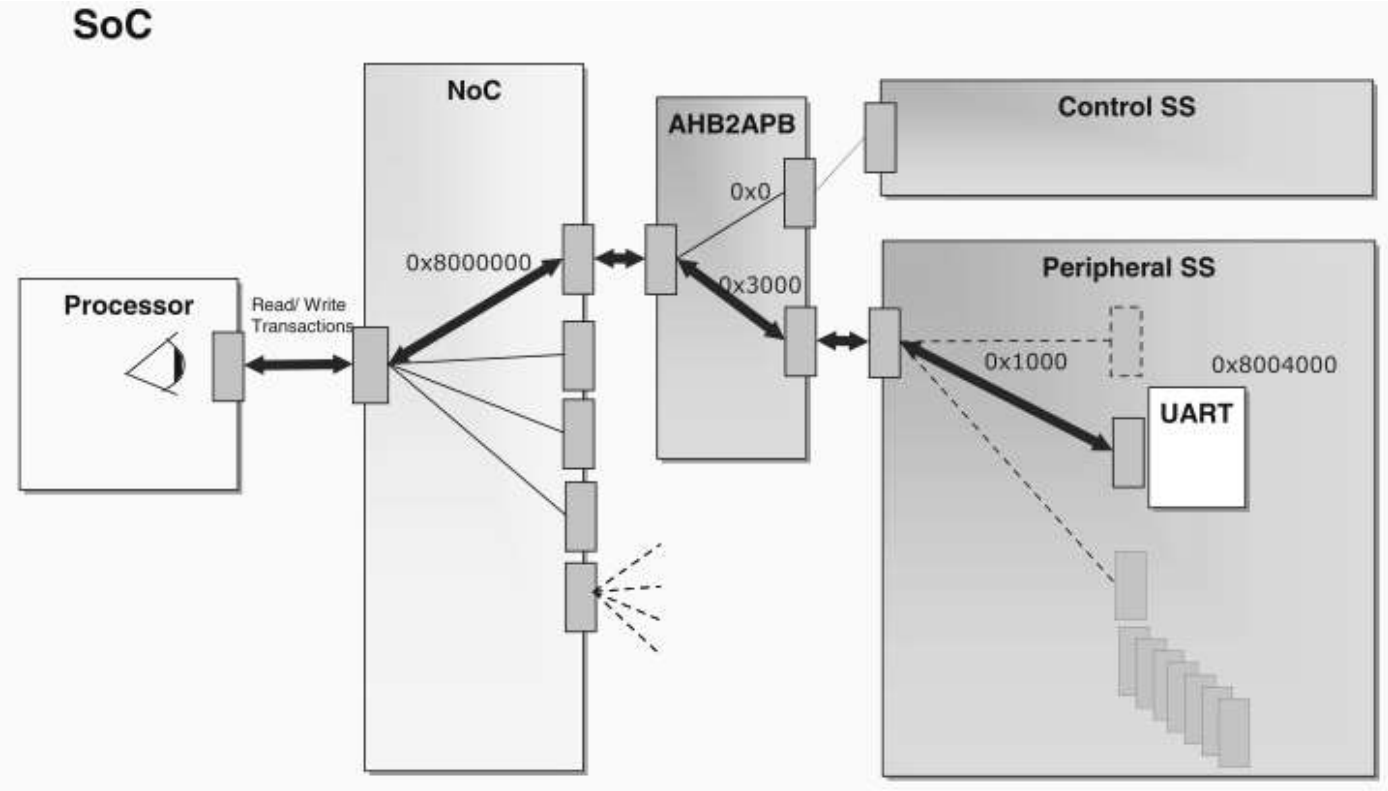
Software API

- Software developers must understand and use **SAHE** to bridge the gap between the hardware and software domains. This is achieved using a **software API** (application programmers interface).
- The **lowest** level of API will typically contain functions for **accessing the SAHEs** in an efficient manner.
- Higher level functions can be created **from** these **low**-level functions to produce a **higher level API**.
- If we consider that a specific hardware IP can be sourced from different IP providers it may be possible to create a level of **abstraction** that would be relatively **hardware-independent**.

Hardware Perspective I

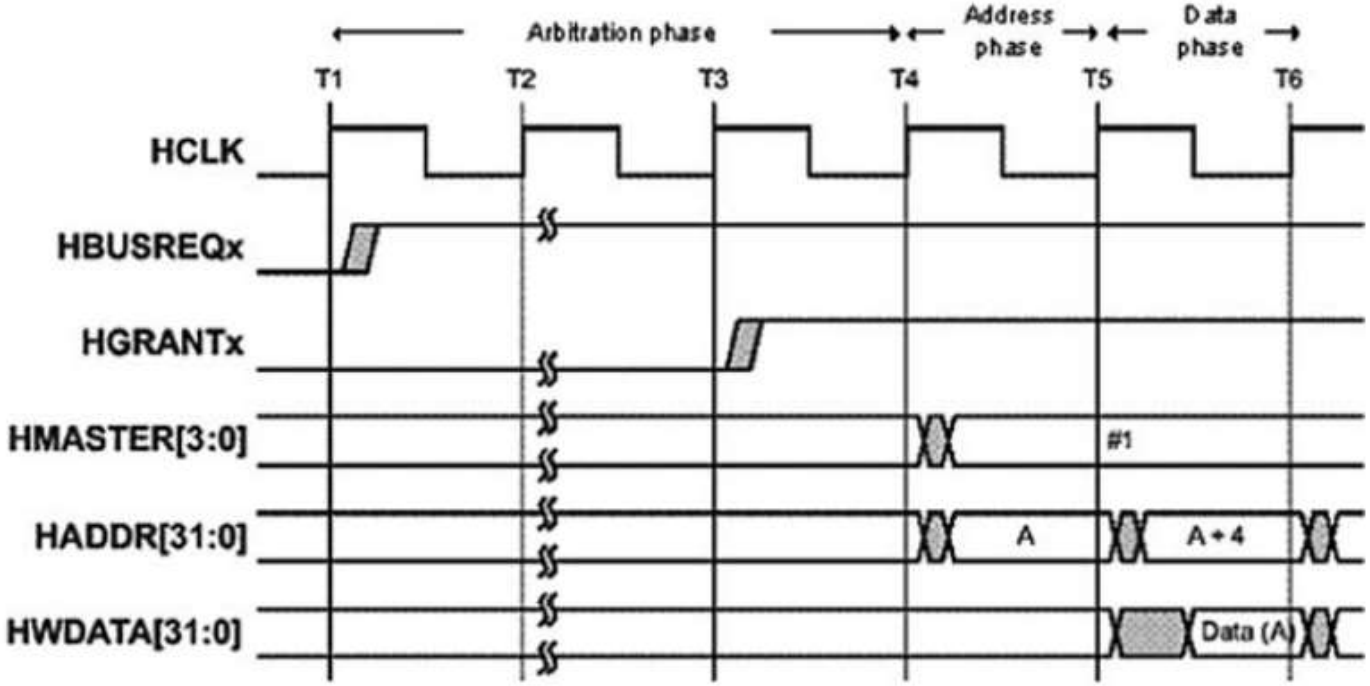
- When software accesses the hardware it does so by means of a **processor transaction**; a read or write access to a specific **address location**.
- This transaction is implemented by a signaling protocol that involves the **toggling of wires** (representing address, data, and control elements) according to a well-defined **specification**.

Hardware Perspective II



Transaction Bus Protocol I

- In the hardware domain, this bus **transaction** is implemented by a signaling protocol involving the **toggling** of wires.



- The protocol is a **specification** of how the transaction is realized in hardware and involves addressing, data, handshaking and control, clocking, and timing; in essence the **logical rules** that govern the communication between the processor and a peripheral.

Transaction Bus Protocol II

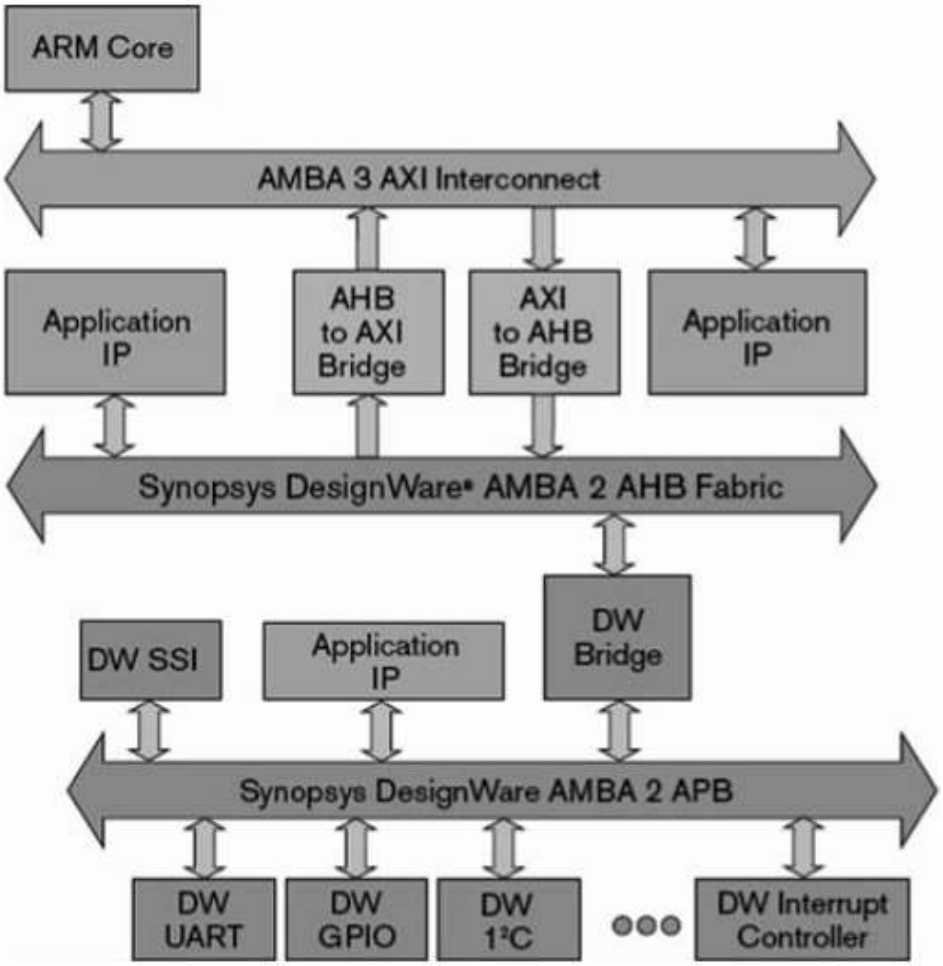
- Examples of protocols include AHB, AXI, PCI, and OCP.
- A bus **master** can be defined as a hardware block that is a capable of **initiating** a data transfer across a system
- A bus **slave** is a logical device capable only of **responding** to a transfer request from a bus master device
- Control options for protocols:
  - Ability to perform **burst** transactions
  - **Interleaved or out-of-order** transactions
  - The **clocking** for the protocol
  - Bus **arbitration** control if there are multiple masters
  - **Byte control** for accessing specific bytes within the read or write data
  - Control needed for master-slave handshaking such as extending transactions by inserting **wait states**
  - Indicating when a write **burst** has been **completed**
  - **Error** control

Transaction Bus Protocol III

- In the hardware system there may be a **mix of protocols** used between the bus master and a specific slave leading to increased complexity through **protocol translation**.

Protocol Translation I

- It is likely in a SoC with IP blocks coming from a range of different sources that a variety of **different protocols** may be used.



Protocol Translation II

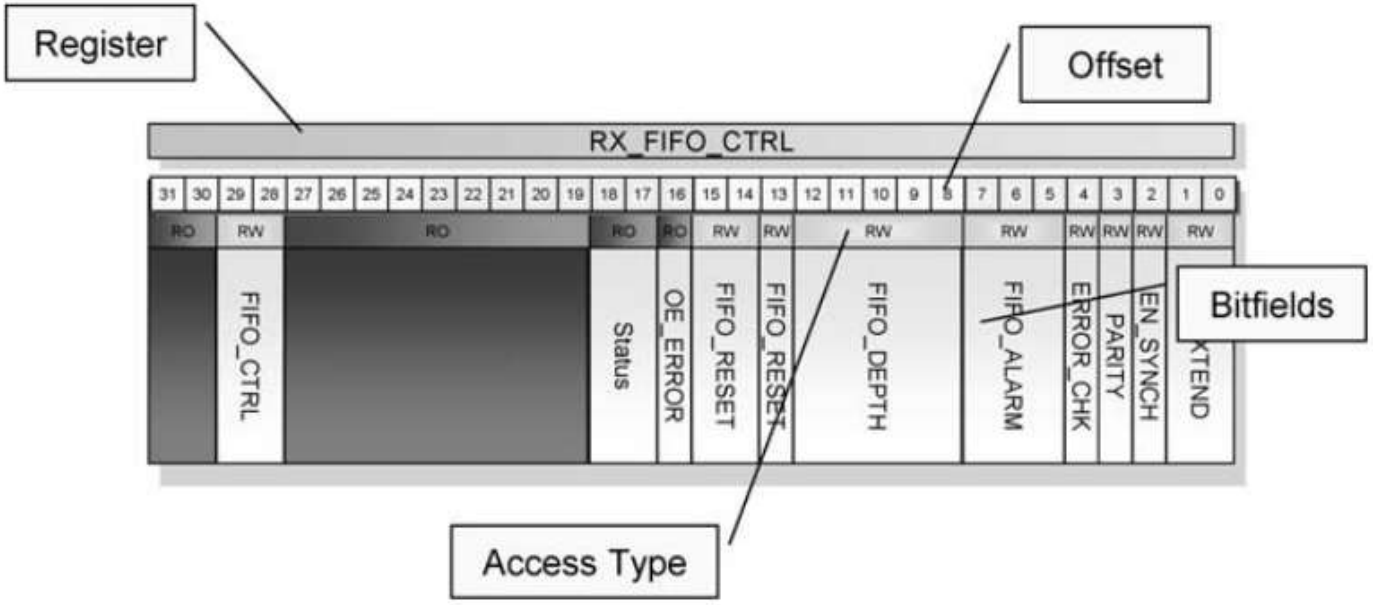
- As a hardware transaction progresses through the system, it may require **several transformations** to other protocols. In addition other transformations may be required, such as:
  - It may need to be **re-synchronized** if the transaction crosses clock domains
  - If the protocol crosses power or voltage domains it may require **isolation cells or level shifters** to be inserted
  - It may need registers inserted to combat **logic delays** or race conditions (also known as **pipelining**)
- All of these transformations and bridging will result in a **delay or latency** between the master initiating a transaction request and the slave's response.
- A general rule of thumb in HW/SW interface design, the aim is **to minimize the number of transactions** required to implement a specific function.

Registers and Bitfields I

- Registers are the common **unit of granularity** in the HW/SW interface
- **Memory-mapped registers** are the **lowest direct level** of HW/SW interface and are usually the last addressable elements in the HW/SW address decoding to have direct read/write communications.
- The difference between a SAHE and a register is that the **SAHE can be any size**, whereas memory-mapped **registers** are constrained by the **LAU** as described earlier.
- The register may have **read-only** access or **read–write** access.
- Because of the volatile nature of hardware it is typical for most writable registers to have a **read back capability**.
- **Bitfields** are considered subsections of the register and can range from a single bit wide to the width of the register.
- Most bitfields have a **one-to-one** correspondence with **SAHEs**

Registers and Bitfields II

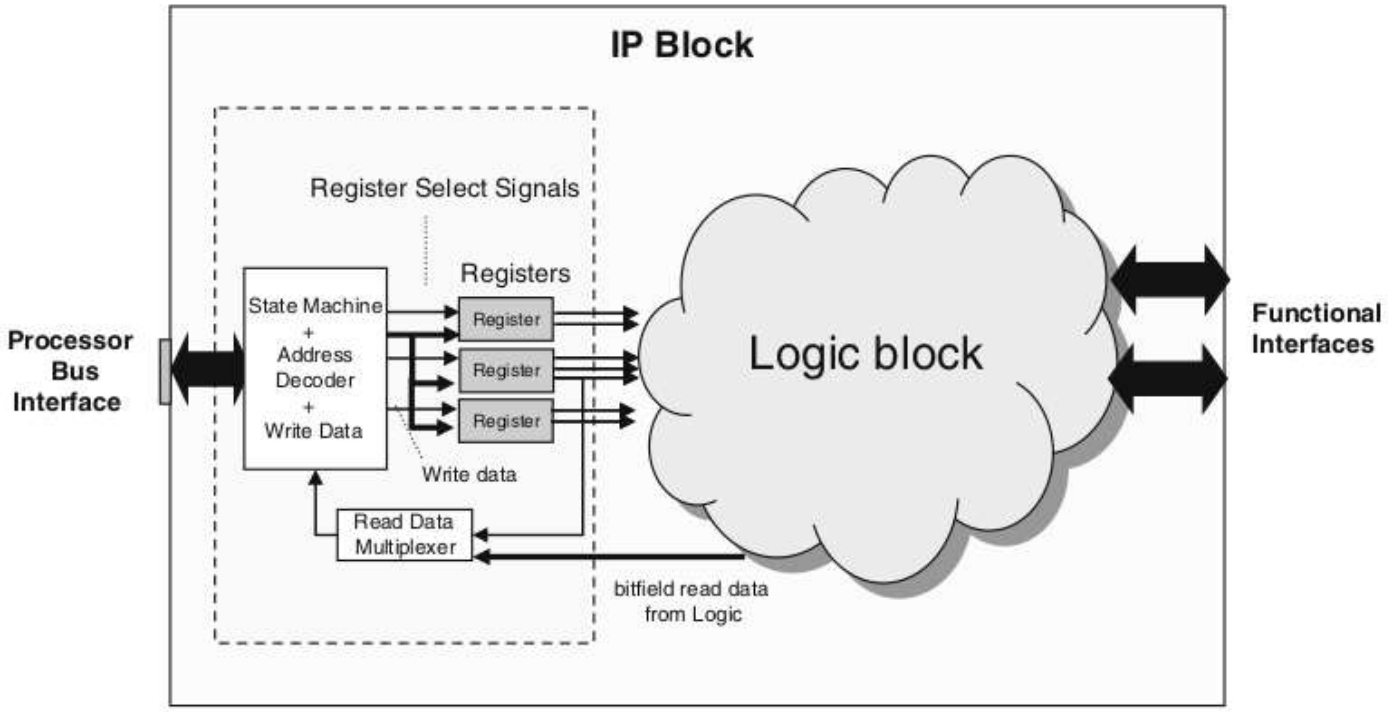
- Bitfields, like SAHEs, can have the following characteristics:
  - The **width** of the bitfield
  - The **offset** of the bitfield within the register
  - The **access type** of the bitfield (e.g., read/write)
  - The value of the bitfield at **reset**.



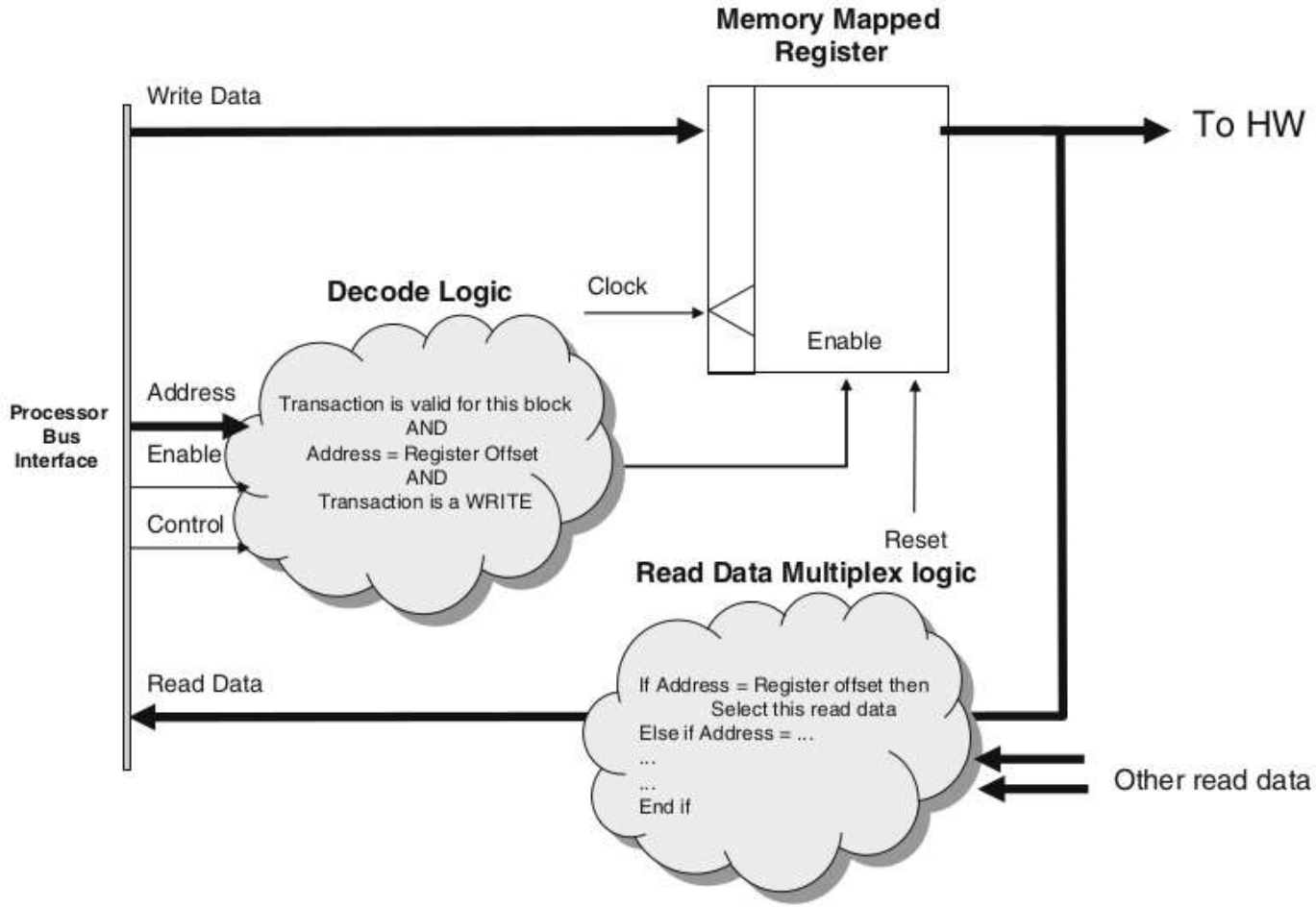
- In order to **write to a register**, the processor must execute an **instruction** that **references** a particular memory-mapped address within the processor address space.

Registers and Bitfields III

- This instruction **initiates** a write **transaction** and the address, control, and write data are **routed through the system** until they arrive at the target IP Block.



Registers and Bitfields IV



- It may be possible to provide a read or write **burst** across the registers and the effects of the transaction **latency** can be **reduced** (averaged).

Registers and Bitfields V

- Another method for reducing transactions is to superimpose additional behavior with the transaction so that a read or write does **more than simply read or write**. This additional behavior is also known as **side effects**.
- **Hardware interrupt status registers** are good examples of side effects.
- Encode an action into the write data to explicitly clear the interrupt bit. This is called a **Write-1-to-Clear (W1C)** type of side effect and setting a particular bit in the data to a 1 will just clear that bit in the register, without affecting any others.
- Further optimization - **Read-to-Clear (R2C)**
- There are a wide range of nuances that exist in the real HW/SW interface including:
  - W1C – Write 1 to clear
  - W0C – Write 0 to clear

Registers and Bitfields VI

  - R2C – Read to clear
  - R2S – Read to set
  - RRR – Read returns remote
- These types of optimizations can challenge **verification** engineers to cover **HW/SW corner cases**, such as ensuring that if a second interrupt assertion happens in the same clock cycle as the access clear, that the interrupt is not lost.
- Also for hardware IP in systems, there are **unlimited possibilities** to what can actually be implemented and corner cases exist in both domains:
  - Registers that are mapped at multiple addresses
  - Registers with one address for read and one for write
  - Read-only registers that mimic a FIFO: every time they are read, a different value is returned

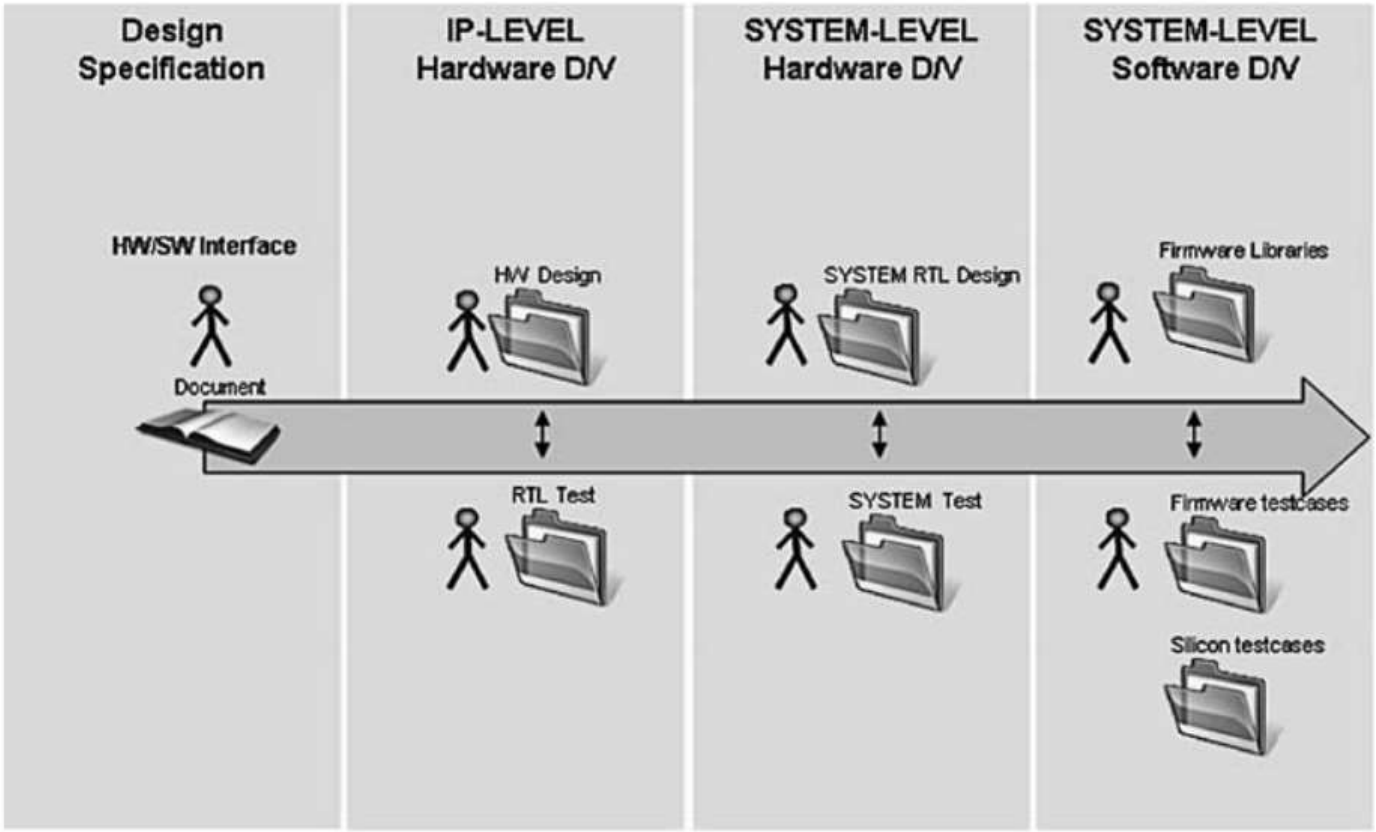
Registers and Bitfields VII

- Registers that change characteristics depending on the state of an internal signal or other register. This is also known as auto-shadow or modal
- Indirect memory access where a set of registers is used to mimic a transaction protocol that is used to access a bigger address space.

Overwiev

HW/SW Design Flow for HW/SW Interfaces I

- The HW/SW interface has traditionally been captured using a **document/paper** specification.
- Traditional HW/SW interface design flow



- Example for the rest of section

HW/SW Design Flow for HW/SW Interfaces II

- SAHE 1 : Reset_Counter : Read/Write, single-bit
- SAHE 2 : Enable_Counter : Read/Write, single-bit
- SAHE 3 : Counter_Value : Read only : 8-bit
- SAHE 4 : Counter_Error : Read only : single-bit.

- An IP designer will start to map these SAHEs into registers.

Specification – Documentation

- The IP owner, or even the system architect, may capture the HW/SW requirements in a **documentation** format.

Register counter_ctrl_status1: COUNTER_CTRL_STATUS

Register Information					
Description	Counter status				
Offset	0x00		Type:	RW	
Bitfield Details					
Field	Name	Description		Access	Default
31:11	Reserved	Not used		RO R returns 0s	0x000000
10	CounterError	Indicates if the counter has an error.		RO	0
		Read 0:	No error		
		Read 1:	FIFO Overflow		
9:2	CounterValue	Value of the counter		RO	0x00
1	EnableCounter	Enables the counter		RW	0
0	ResetCounter	resets the counter		RW	0
		0:	counter it not reset		
		1:	Reset the counter. This values hould be cleared again in order for counter to start		

IP-XACT (SPIRIT) I

- A **subset** of IP-XACT can be used to specify memory maps, register, and bitfield details.

```
...
<spirit:register>
  <spirit:name>counter_ctrl_status</spirit:name>
  <spirit:dim>1</spirit:dim>
  <spirit:addressOffset>0</spirit:addressOffset>
  <spirit:size spirit:resolve = "user">32</spirit:size>
  <spirit:access>read-write</spirit:access>
  <spirit:reset>
    <spirit:value>0</spirit:value>
    <spirit:mask>4294967295</spirit:mask>
  </spirit:reset>
  <spirit:field>
    <spirit:name>ResetCounter</spirit:name>
```

IP-XACT (SPIRIT) II

```
    <spirit:bitOffset>0</spirit:bitOffset>
    <spirit:bitWidth spirit:resolve = "user">1</spirit:bitW
    <spirit:access>read-write</spirit:access>
    <spirit:description>resets the counter</spirit:descript
    <spirit:values>
      <spirit:value>0</spirit:value>
      <spirit:description>counter it not reset</spirit:desc
      <spirit:name>No_effect</spirit:name>
    </spirit:values>
    <spirit:values>
      <spirit:value>1</spirit:value>
      <spirit:description>Reset the counter. This value . .
      <spirit:name>reset_counter</spirit:name>
    </spirit:values>
  </spirit:field> ...
```

SystemRDL I

- The **SystemRDL** language, supported by the SPIRIT Consortium, was specifically designed to describe and implement a wide variety of control status registers.

```
reg { // REGISTER
// ADDRESSOFFSET 0x0
// ACCESS read-only
    name = "counter_ctrl_status";
    desc = "Counter status";
    regwidth = 32;
    default hw = rw; default sw = rw;
    field {
name = "ResetCounter";
desc = "resets the counter";
hw = rw; sw = rw;
fieldwidth = 1;
```

SystemRDL II

```
    } ResetCounter [0:0];
    field {
name = "EnableCounter";
desc = "Enables the counter";
hw = rw; sw = rw;
fieldwidth = 1;
    } EnableCounter [1:1];
    ...
} counter_ctrl_status @0x0; // END REGISTER
```

IP Hardware Design

- An RTL model of the HW/SW interface requires **logic** that **translates** the bus protocol into read and write access to specific registers or memories.
  - Address decoding to enable write and read accesses
  - A model of the register.

IP Verification

- From an IP (block level) point of view, HW/SW interface verification is typically done using **simulation-based approaches**.
- One of the main verification objectives is to **exhaustively** test the **write and read accesses** to the IP block
  - Testing the address decode to ensure that not only are all the registers at the correct address but that accessing **invalid** addresses has **no effect**
  - Testing the **reset** value of the registers
  - Testing that all valid bits of a register can be **accessed**.
  - Testing that all **side effects** behave as specified.
- Typically **UVM** is used

Chip-Level Verification I

- Within a full chip-level view, the IP now has a **system-level context** and sits somewhere in a memory map
- The following are some typical errors in the HW/SW interface
  - **Inconsistencies** between a register **specification** and its corresponding **implementation** including bitfield layout, register accesses, and reset values
  - Address mirroring of registers because internal address **buses** are defined to be too **small**
  - Incorrect address **decoding**
  - Incorrect memory map specification causing **overlaps**.
- Because of the potential for errors and mismatches, the low-level HW/SW interface therefore still needs a **strong focus**, even at chip level.
- At chip-level the HW/SW access can be tested between the origin of the transaction (the **processor interface**) and the **transaction target**

Chip-Level Verification II

- Chip-level **testbenches**, modeling environments, and test-case format can be **very different**.
  - Test cases can be written using a **HVL** to emulate the processor transactions by focusing on read/write transactions
  - The test cases may also be written in **C and compiled as binaries** which are then run on a **model** of the embedded **processor**.
- In order to achieve an acceptable level of coverage in the HW/SW interface, it is important that registers are modeled from **the perspective of the processor**.
- This type of simulation-based methodology will yield **poor simulation times** in an RTL simulator environment.
- Another interesting approach is with **formal verification** which aims to prove functional correctness through formal methods, requiring no simulation.

Software Development – Firmware I

- The lowest level SW/HW interaction needs **efficient access** to the SAHEs and can do so by means of an **API**.
- Essentially they are about **reading and writing** to a memory map.
- Can be coded an infinite **number of ways**.
- `* ((volatile uint32_t * )0xffee0000) = 8;`
- Easy, but hard to maintain.
- ```
  #define EVENTMONITOR_COUNTER_CTRL_STATUS 0xffee0000
  * (volatile uint32_t * ) EVENTMONITOR_COUNTER_CTRL_STATUS
                            = 8;
  ```
- ```
  #define EVENTMONITOR_COUNTER_CTRL_STATUS \
      ((volatile uint32_t * ) 0xffee0000)
  * EVENTMONITOR_COUNTER_CTRL_STATUS = 8;
  ```
- The Register model could also be described in a **variety** of different **C++** formats

Software Development – Firmware II

- **Functions** are often used to encapsulate the low-level accesses and when used **with macros** can be a **good general method**.

```
#define EVENTMONITOR_COUNTER_CTRL_STATUS_ENABLECOUNTER_MASK (U32T)0x00000002u
#define EVENTMONITOR_COUNTER_CTRL_STATUS_ENABLECOUNTER_OFFSET 1

extern void EventMonitor_WriteCounter_ctrl_status_EnableCounter (
    const U32T baseAddress,
    const U32T value)
{
    const U32T offset = baseAddress + EVENTMONITOR_COUNTER_CTRL_STATUS_OFFSET;
    register U32T data = RD_MEM_32_VOLATILE(offset);
    register U32T newValue = value;
    data &= ~(EVENTMONITOR_COUNTER_CTRL_STATUS_ENABLECOUNTER_MASK);
    newValue <<= EVENTMONITOR_COUNTER_CTRL_STATUS_ENABLECOUNTER_OFFSET;
    newValue &= EVENTMONITOR_COUNTER_CTRL_STATUS_ENABLECOUNTER_MASK;
    newValue |= data;
    WR_MEM_32_VOLATILE(offset, newValue);
}
```

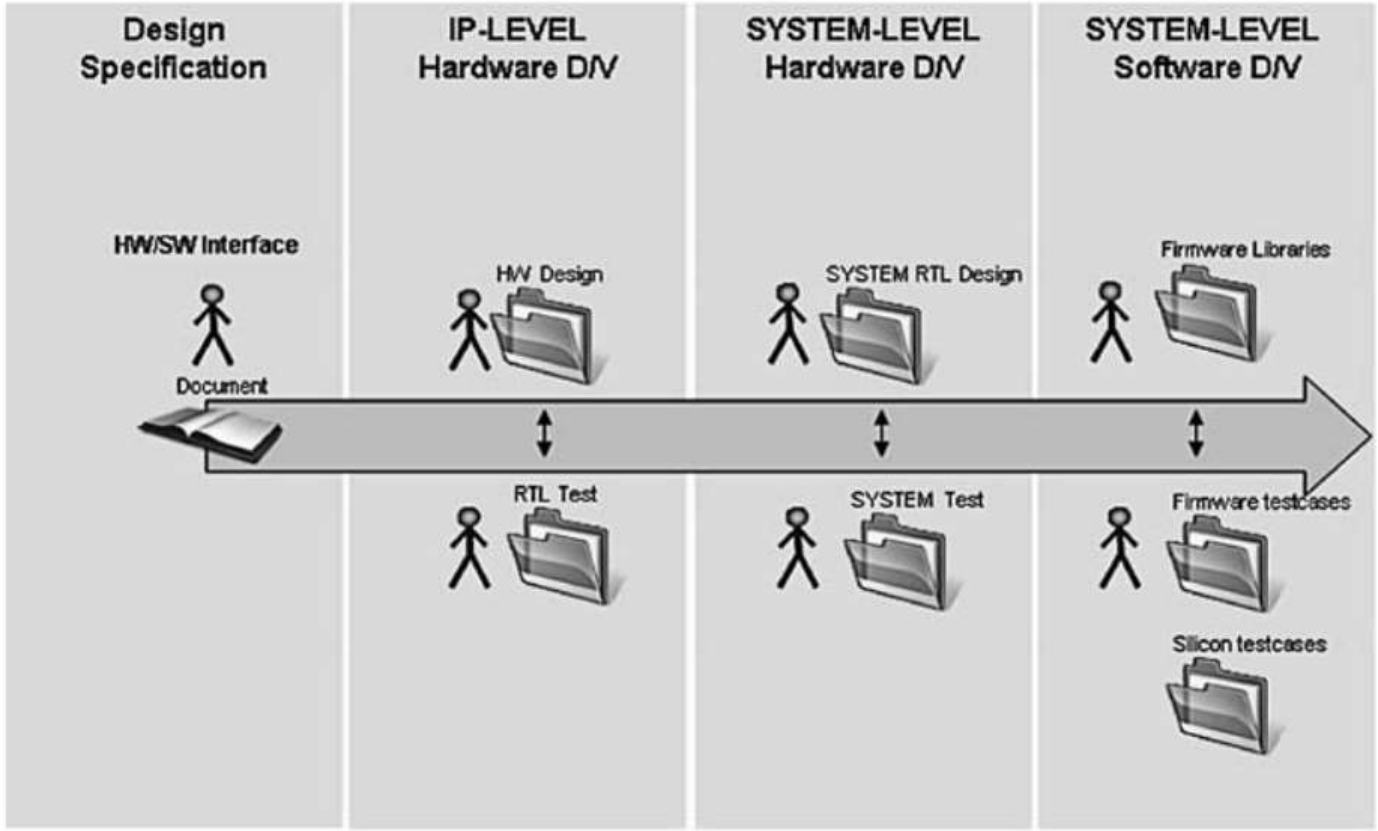
Firmware Verification

- Software can be developed and tested against a **model** of the hardware.
- The hardware model can be of **different formats**. It can range from an **RTL** format which is a very accurate low-level hardware model to a software model of the hardware, also called **virtual models**.
- Depending on the software development criteria there are a range of different models with different strengths and weaknesses.

Overwiev

HW/SW Interface Tools and Design Flows I

- The HW/SW interface has traditionally been captured using some form of a **document** specification.



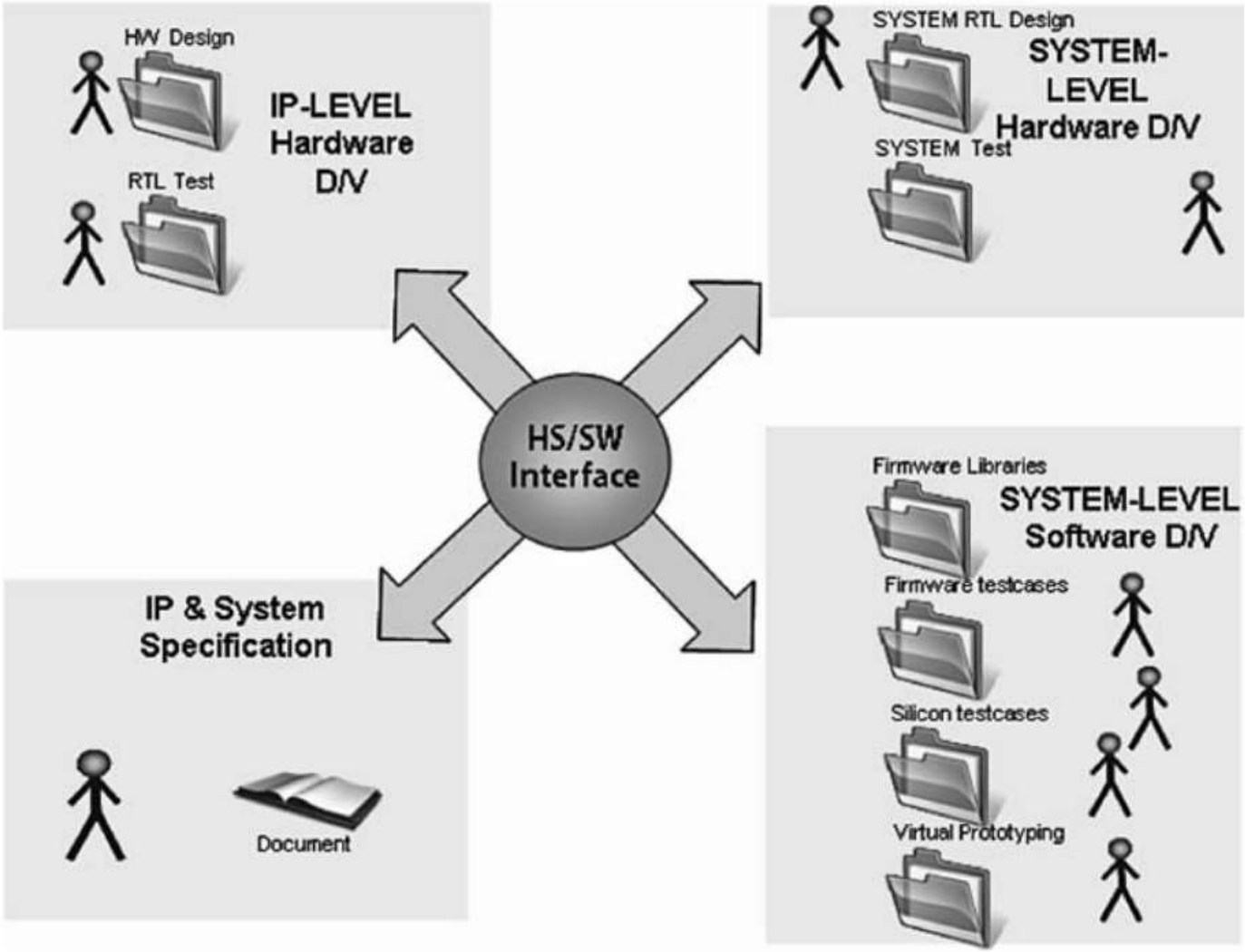
HW/SW Interface Tools and Design Flows II

- All HW/SW interface related teams need to **interpret** the specification and **translate** it to their native format. It may be possible for them to give **feedback** where a change is required in the document.
- Interpret–Translate–Feedback (ITF)
- If the document is open to **interpretation** the **wrong** information may get translated and cause a mismatch somewhere in the design flow.
- A document-based process will introduce **interpretation bugs**, **synchronization** bugs, and **translation** bugs at various stages in the design flow.

Register Management Tools I

- There are new tools emerging in the market that address this problem through **automated**, **correct-by-construction** methodologies dedicated to the HW/SW interface.
- The main concept behind the solutions is to move away from using documents to an **executable specification**.

Register Management Tools II



- Main benefits of tools:
  - A user interface enabling efficient capture

Register Management Tools III

- Usable from all perspectives - HW/SW/design/verification/documentation
- An extensive **underlying model** of the HW/SW interface
- Extensive coherency checking at all levels
- IP/sub-system and system capture and manipulation
- Interfaces to **standards** like IP-XACT
- Full **regression** capability to allow overnight builds of the HW/SW interface
- Flexible generators to **generate** a wide range of HW/SW design verification collateral including.
  - Documentation - programming reference guide
  - RTL code for modeling registers
  - HVL code for IP and chip-level testbenches.
  - SystemC for virtual prototyping
  - Firmware header files.

Overwiev

IP Databases I

- The IP revolution had barely begun in the mid-1990s, when people began looking at the conversion of unstructured datasheet information (or its web equivalents) into **more formal databases** for IP components or blocks.